A Framework for automated selective Fine-Tuning of Domain-Specific Large Language Models Using Graph-Based Retrieval Augmented Generation

Hariharan Govindharajan[1],
Senthilkumar Vijayakumar[2]

{[1]hariharang,[2]senthil.vijayakumar}@ieee.org

2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON) | 979-8-3315-4090-6/24/$31.00 ©2024 IEEE | DOI: 10.1109/UEMCON62879.2024.10754778

***Abstract* - Graph based retrieval augmented generation technique in Large Language Model (LLM) brings in major advantages by providing deep context to LLMs through relational knowledge graph for text generation, classification, question and answering use cases. However, maintaining vast data volume of domain specific data in a knowledge graph with complex relationships and querying from it every time a prompt is being posted to LLM, is a time consuming and expensive process. This paper presents a novel framework for selectively fine-tuning domain-specific large language models (LLMs) using a multi-stage Knowledge Graph (KG) based Retrieval Augmented Generation (RAG) pipeline and an Automated Incremental Fine-tuning System (AIFS). The proposed system aims to enhance the accuracy and relevance of LLM responses for text generation and Question Answering use cases by fine-tuning the LLM incrementally based on highly sought and highly relevant information in knowledge graph identified by leveraging page rank algorithm in KG. The framework comprises three major subsystems: Knowledge Graph Generation, Automated Incremental fine-tuning system (AIFS), and Domain Based Information Retrieval (DBIR). The effectiveness of the system is demonstrated through its ability to incrementally fine-tune LLMs based on selected highly relevant nodes within the KG, thereby improving the model's domain-specific knowledge, response accuracy by 90% and reduce cost by 71.8%.**

Keywords—Artificial Intelligence(AI),Knowledge Graph(KG), Large Language Models(LLM), LLM Fine-tuning, Contextual Information Extraction & Retrieval Systems

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has revolutionized natural language processing tasks across various domains using knowledge graphs [1][2] and Retrieval Augmented Generation (RAG)[16]. However, adapting these KG based models to specific domains while maintaining efficiency and accuracy with cost control remains as a significant challenge [3]. Generally, either a KG based RAG or fine-tuning approach is followed to adapt the model to domain specific tasks. But both have their own pros and cons. If RAG is followed to adapt the domain specific tasks, it could increase the cost by increasing the number of tokens to be sent each time to LLM to provide contextual knowledge. This is especially more expensive for highly relevant and highly sought information as we have to store complex relationships in KG and send same tokens to LLM for context redundantly for similar queries from different users. Fine tuning on the other hand, could increase cost and time as in case of huge amount of data that needs to be trained. This paper presents a novel framework that addresses this challenge by combining graph-based Retrieval-Augmented Generation (RAG) and an automated selective fine-tuning process with highly sought information from KG for domain-specific LLMs.

The proposed framework consists of three main components: a Knowledge Graph Generation, an Automated Incremental Fine-tuning System (AIFS), and a Domain Based Information Retrieval mechanism (DBIR). This integrated approach aims to enhance the performance of LLMs in domain-specific applications by leveraging knowledge graphs and implementing an automated fine-tuning process based on highly sought and relevant information.

The Knowledge Graph Generation component focuses on information extraction and knowledge graph generation, providing a structured representation of domain-specific knowledge. The AIFS utilizes the page rank algorithm[14][15] of the knowledge graph to identify highly relevant and sought nodes and fine-tune the LLM accordingly. Finally, the Domain-based Information Retrieval component ensures efficient query processing by utilizing the fine-tuned model and supplementing it with additional information from the knowledge graph when necessary.

This paper will explore each component of the framework in detail, discussing their underlying mechanisms, interactions, and the advantages they offer in developing and maintaining domain-specific LLMs. It also follows an approach of taking the below example and walk it through the process of explaining how to build the framework to achieve the desired results

"NIST cloud computing definition, a cloud infrastructure may be operated in one of the following deployment models: public cloud, private cloud, community cloud, or hybrid cloud. For the details related to the controls and management in the cloud, we refer the readers to the NIST Special Publication 800-146, NIST Cloud Computing Synopsis and Recommendations"

II. KNOWLEDGE GRAPH GENERATION

The framework for generating a Knowledge Graph (KG) in specialized domains is a systematic approach to transforming unstructured data into structured, interconnected entities and relationships. The core process, known as the Knowledge Graph Generation, encompasses several stages, including Information Extraction, Triples Extraction, and Ontology

979-8-3315-4090-6/24/$31.00 ©2024 IEEE

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

Linking, culminating in the creation of a cohesive KG. This KG serves as a powerful resource for enhancing the capabilities of Large Language Models (LLMs) by providing domain-specific, structured knowledge.

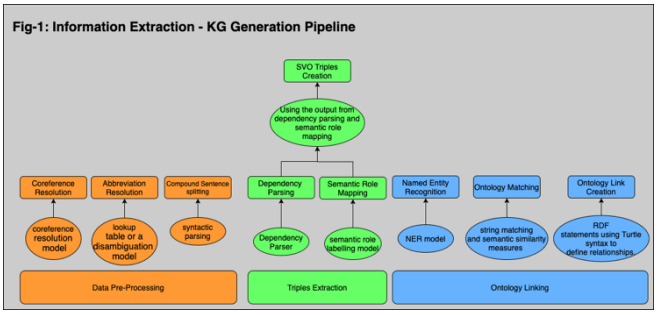


Figure-1: Knowledge Graph Generation

A. *Information Extraction – KG Generation Pipeline*

The KG generation pipeline involves meticulous steps designed to refine unstructured data into a format suitable for integration into a KG. This process includes Data Preprocessing, Triples Extraction, and Ontology Linking. Refer Figure-1.

1) *Data Preprocessing*

Objective: To sanitize and standardize unstructured text data, ensuring consistency and clarity for subsequent processing.

a) Coreference Resolution:

- **Definition:** Coreference resolution involves identifying all expressions in a text that refer to the same entity.
- **Technical Process:** Utilize a coreference resolution model to replace pronouns and synonyms with the primary entity reference. For example, in the sentence "A cloud infrastructure may be operated in one of the following deployment models: public cloud, private cloud, community cloud, or hybrid cloud," instances of "it" referring to "cloud infrastructure" are replaced with "cloud infrastructure."
  - **Equation:** If $\boldsymbol{p_i}$ is a pronoun and $\boldsymbol{e_j}$ is the antecedent, then the resolution process can be represented as:

$$\boldsymbol{p_i} \rightarrow \boldsymbol{e_j}$$

b) Abbreviation Resolution:

- **Definition:** The process of expanding abbreviations and acronyms to their full forms to ensure clarity.
- **Technical Process:** Implement a lookup table or a disambiguation model that maps abbreviations to their full forms. For instance, "NIST" is resolved to "National Institute of Standards and Technology."
- **Example:** NIST→National Institute of Sta ndards and Technology

c) Compound Sentence Splitting:

- **Definition:** Decomposing complex sentences into simpler ones to facilitate accurate entity and relationship extraction.
- **Technical Process:** Use syntactic parsing to identify and split compound sentences. For example, "A cloud infrastructure may be operated in one of the following deployment models: public cloud, private cloud, community cloud, or hybrid cloud" is split into separate sentences, each focusing on a specific deployment model.
- **Example:** "A cloud infrastructure may be operated in a public cloud model." "A clou d infrastructure may be operated in a privat e cloud model."

2) *Triples Extraction*

Objective: To extract Subject-Verb-Object (SVO) triples from the preprocessed text, providing a structured representation of the semantic content.

a) Dependency Parsing:

- **Definition:** Analyzing the grammatical structure of a sentence to understand relationships between words.
- **Technical Process:** Implement a dependency parser that maps out syntactic dependencies between words. For instance, in the sentence "The cloud infrastructure may be operated in a hybrid cloud model," the parser identifies "cloud infrastructure" as the subject, "operated" as the verb, and "hybrid cloud model" as the object.
- **Equation:** Dependency relation R can be represented as:

  R(subject,verb,object)=(cloud infrastru cture,operated,hybrid cloud model)

b) Semantic Role Mapping:

- **Definition:** Assigning semantic roles to entities in a sentence to understand their functions within a context.
- **Technical Process:** Use a semantic role labeling model to assign roles such as agent, action, and target. In the sentence "The cloud infrastructure may be operated in a hybrid cloud model," "infrastructure" is the agent, "operated" is the action, and "hybrid cloud model" is the target.
- **Equation:** Semantic roles can be represented as:

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

Roles(agent,action,target)=(cloud infra structure,operated,hybrid cloud model)

c) SVO Triples Creation:

- **Definition:** Creating structured representations (triples) from extracted data.
- **Technical Process:** Using the output from dependency parsing and semantic role mapping, construct SVO triples. For example, the triple ("infrastructure", "operated", "hybrid cloud model") represents the entity-action-target relationship.
- **Example:** (infrastructure,operated,hybrid c loud model)

3) Ontology Linking

Objective: To align extracted triples with a predefined domain-specific ontology, ensuring semantic coherence.

a) Named Entity Recognition (NER):

- **Definition:** Identifying and categorizing key entities in the text.
- **Technical Process:** Utilize an NER model to detect and classify entities. For example, "cloud infrastructure" and "NIST" are identified as technical terms related to cloud computing.
- **Example:** NER(cloud infrastructure)=Tec hnical Term

b) Ontology Matching:

- **Definition:** Associating identified entities with concepts in a domain-specific ontology.
- **Technical Process:** Use string matching and semantic similarity measures to link entities with ontology concepts. For instance, mapping "public cloud" to the corresponding concept in a cloud computing ontology.
- **Equation:** Ontology matching can be represented as: Match(entity,ontology concept)=Match (public cloud,CloudDeploymentModel.Pu blic)

c) Ontology Link Creation:

- **Definition:** Establishing formal relationships between entities and ontology concepts using Resource Description Framework (RDF).
- **Technical Process:** Generate RDF statements using Turtle syntax to define these relationships. For instance, creating a statement like "cloud infrastructure is-a deployment model."
- **Example:** @prefix cc: <http://example.org/cloudcomputing#> cc:cloud_infrastructure cc:DeploymentModel .

B. Knowledge Graph Creation

The second phase involves assembling the extracted and linked entities, relationships, and ontology concepts into a structured KG.

1) Ontology Selection:

- **Objective:** Choose an appropriate ontology that encapsulates the domain knowledge.
- **Technical Process:** Select a well-defined ontology, such as a cloud computing ontology, that provides a taxonomy and relationships for domain-specific concepts.
- **Example:** Selecting the Cloud Computing Ontology (CCO) for cloud infrastructure data.

2) Entity Mapping:

- **Objective:** Accurately map named entities to ontology concepts.
- **Technical Process:** Cross-reference extracted entities with ontology entries to ensure correct representation.
- **Example:** Mapping "NIST" to the ontology concept "StandardOrganization."

3) Relation Mapping:

- **Objective:** Define and map relationships between entities in accordance with the ontology.
- **Technical Process:** Use the ontology to determine and assign relationships, ensuring they are represented accurately within the KG.
- **Example:** Mapping the relationship "operated in" to the ontology concept "hasDeploymentModel."

4) RDF Link Creation:

- **Objective:** Create Resource Description Framework (RDF) links that formalize the relationships between entities and ontology concepts.
- **Technical Process:** Use Turtle syntax to define RDF statements that encapsulate these relationships. This involves creating triples that define how entities are related within the context of the ontology.
- **Example:** cc:cloudinfrastructurecc:hasDep loymentModelcc:HybridCloud.

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

5) *Graph Creation:*

- **Objective:** Assemble the knowledge graph, incorporating all linked entities, relationships, and RDF links.
- **Technical Process:** Integrate all components into a cohesive graph structure using a graph database or a KG platform. This graph serves as a comprehensive network of domain-specific knowledge.
- **Example:** A graph visualization showing entities like "cloud infrastructure," "NIST," and relationships such as "hasDeploymentModel."

This section outlined the detailed process for generating a Knowledge Graph from unstructured text data in specialized domains. By leveraging advanced NLP techniques such as coreference resolution, dependency parsing, and ontology linking, the framework systematically converts text into a structured, semantically rich KG. This KG not only enhances the capabilities of LLMs but also facilitates advanced applications like information retrieval, text classification, and question answering in domain-specific contexts. The use of domain-specific ontologies ensures that the extracted knowledge is precise and relevant, making the KG a valuable asset for various specialized applications.

.

III. Automated Incremental Fine-tuning System (AIFS) for Domain-Specific Language Models

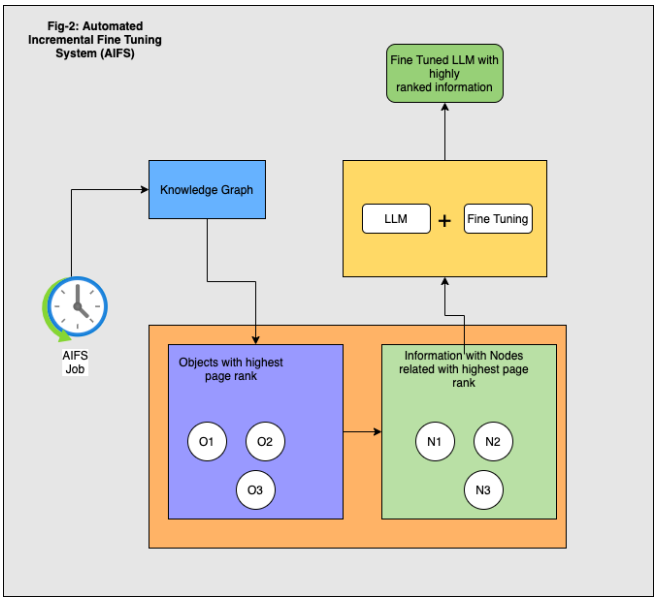


Figure-2: Automated Incremental Fine-Tuning System (AIFS)

The Automated Incremental Fine-Tuning System (AIFS) is designed to enhance the domain-specific Large Language Models (LLMs) using the structured knowledge represented in a Knowledge Graph (KG) generated by the Knowledge Graph Generation pipeline. AIFS provides a dynamic mechanism to incrementally improve the LLM knowledge by iteratively fine-tuning it with the most relevant and frequently accessed domain-specific information. Refer Figure-2. This system operates through several critical steps, each of which is elaborated below with additional technical details and considerations.

A. *Page Rank Analysis*

Objective: Identify and prioritize the most relevant information nodes within the KG for fine-tuning the LLM.

The KG is represented as a graph G=(V,E), where V is the set of nodes (entities) and E is the set of edges (relationships). The PageRank algorithm [14], commonly used to measure the importance of nodes, is applied to the KG to rank the nodes based on their connectivity and relevance.

The PageRank score for a node v is defined as:

$$\mathrm{PR(v)} = \frac{1-d}{N} + d \sum_{u \in M(v)} \frac{PR(u)}{L(u)}$$

where:

- PR(v) is the PageRank score of node v,
- d is the damping factor,
- M(v) is the set of nodes linking to v,
- PR(u) is the PageRank of node u
- N is the total number of pages
- L(u) is the number of outbound links from node u.

Nodes with higher PageRank scores represent information that is frequently accessed or deemed highly relevant within the domain, as illustrated in the context of Eg 1: “NIST cloud computing definition” and associated concepts like "public cloud," "private cloud," etc.

B. *Highly sought Information Extraction*

Objective: Extract and prepare textual information from high-ranking nodes for the fine-tuning process.

The content associated with high-ranking nodes, identified through PageRank analysis, is extracted. The ideal number of high-ranking nodes to consider when using PageRank in a knowledge graph is context-dependent and should be determined based on the specific goals, network characteristics, and available resources. This includes not only the textual descriptions of these nodes but also the context in which they appear within the KG. For example, if the node "public cloud" has a high PageRank, the system extracts the definition, characteristics, and related concepts, as outlined in the Eg 1 scenario, which includes references to NIST Special Publication 800-146.

C. *Model Fine-Tuning*

Objective: Fine-tune the LLM with the extracted information to enhance its domain-specific capabilities.

Fine-tuning involves updating the LLM's parameters to improve its performance in a specific domain. The extracted information D={d1,d2,…,dn} serves as the fine-tuning dataset, where each di represents a document or text snippet related to the domain.

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

The fine-tuning process is formalized as optimizing the model's parameters θ to minimize the loss function L over the dataset D:

$$\theta * = \arg\min_{\theta} \sum_{i=1}^{n} L(f\theta(di), yi)$$

where

- f_θ is the large language model (LLM) with parameters θ
- d_i is an input text from the dataset
- y_i is the ground truth or target output

For the domain-specific context, the loss function may include additional regularization terms to account for domain-specific knowledge constraints, ensuring the model's outputs align with domain norms and standards.

D. Knowledge Graph Update

Objective: Update the KG by removing the fine-tuned nodes, ensuring efficient handling of future queries.

After fine-tuning, the system removes the corresponding high-ranking nodes from the KG. This step is crucial to prevent redundancy and ensure that the fine-tuned LLM handles future queries directly, bypassing the KG for previously fine-tuned information. This mechanism also allows the system to focus on fine-tuning the model further with new highly relevant and frequently accessed information, continuously expanding the LLM's knowledge base.

The node removal can be mathematically represented as:

$$G' = (V - V_{fine-tuned}, E - E_{fine-tuned})$$

where

$V_{fine-tuned}$ and $E_{fine-tuned}$ are the sets of nodes and edges associated with the fine-tuned information, respectively.

E. Model Deployment

Objective: Deploy the updated LLM version in the live environment for real-time query handling.

The newly fine-tuned model, now encapsulating the most relevant and updated domain knowledge, is versioned and deployed. The deployment process involves rigorous validation to ensure that the updated model maintains or improves performance metrics such as accuracy, response time, and relevance.

The deployment can be formalized as transitioning from model version M_{prev} to M_{new}, ensuring backward compatibility and seamless integration with existing systems:

$$M_{new} = M_{prev} + \Delta M$$

where ΔM represents the incremental improvements made during the fine-tuning process.

The AIFS provides a robust and automated framework for the continual improvement of domain-specific LLMs. By leveraging the structured information within a KG and employing systematic fine-tuning and updates based on high ranking nodes, the AIFS ensures that the LLM remains accurate, relevant, and up-to-date. This dynamic adaptation to the most frequently accessed and pertinent information enhances the model's performance and efficiency, making it a powerful tool for specialized applications such as information retrieval, text classification, and domain-specific question answering.

IV. Domain-Based Information Retrieval

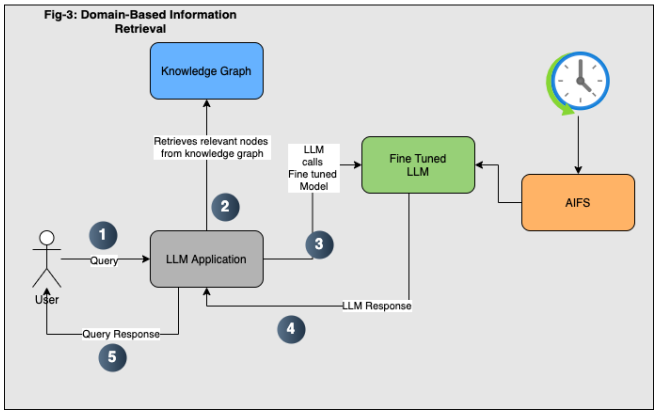


Figure-3: Domain Based Information Retrieval

The Domain-Based Information Retrieval (DBIR) component is an integral part of the knowledge extraction framework, leveraging the synergy between the fine-tuned Large Language Model (LLM) and the Knowledge Graph (KG) to deliver comprehensive and precise responses to user queries. Refer Figure-3. This component is designed to optimize response accuracy and relevance by utilizing both the updated LLM and the structured data within the KG. The operational workflow of DBIR is delineated into three key stages, detailed with technical nuances and contextual examples.

A. Query Processing

Objective: Efficiently interpret and process user queries using the fine-tuned LLM.

When a user submits a query Q, it is first parsed and preprocessed to extract the essential elements that can be used for matching with the knowledge base. The fine-tuned LLM, which has been incrementally updated with domain-specific knowledge, is employed to process this query.

Let Q be represented as a sequence of tokens {q1,q2,…,qn}. The LLM f_θ processes this sequence to generate a contextualized vector representation h_Q for the query:

$$h_Q = f_\theta(Q)$$

where h_Q encodes the semantic meaning of the query based on the LLM's internal knowledge.

For instance, if the query pertains to "public cloud," the LLM, fine-tuned on domain-specific data like Eg 1 ("NIST cloud computing definition..."), can generate a highly relevant and accurate response without further retrieval.

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

B. *Supplementary Retrieval*

Objective: Retrieve supplementary information from the KG when the query involves low ranked nodes tha are not yet used for fine-tuning.

In cases where the query Q pertains to information outside the fine-tuned scope of the LLM or requires more detailed data, the system initiates supplementary retrieval from the KG. This involves querying the KG database G=(V,E), where nodes V represent entities and edges E represent relationships.

The KG is queried using a semantic similarity measure to match the query vector h_Q against the node embeddings in the KG. The similarity score $S(v,h_Q)$ for each node $v \in V$ is computed using a function such as cosine similarity:

$$s(v, h_Q) = \frac{h_v . h_Q}{\|h_v\| h_Q\|}$$

where h_v is the embedding of node v.

The top k nodes with the highest similarity scores are retrieved, where k is a predefined threshold:

$$V_{retrieved} = \{ v \in V \mid s(v, h_Q) \text{ is amoing top } k$$

This supplementary information retrieval ensures comprehensive coverage, particularly for queries about less frequent or emerging topics not covered in the LLM's fine-tuning phase. For example, queries about specific deployment models or security measures in cloud computing can be supplemented with detailed descriptions from NIST Special Publications, as highlighted in Eg 1.

C. *Response Generation*

Objective: Synthesize a comprehensive and accurate response by combining information from the LLM and KG.

The final response is generated by the LLM, incorporating both its internal knowledge and the supplementary information retrieved from the KG. The LLM uses its response generation mechanism, which may include techniques like sequence-to-sequence modeling or attention mechanisms, to integrate and articulate the relevant data.

Let R_{LLM} be the initial response generated by the LLM based solely on its internal knowledge, and R_{KG} be the additional information retrieved from the KG. The final response R_f is composed as:

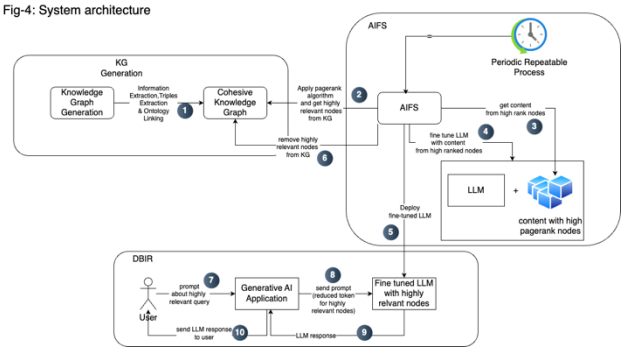
$$R_f = R_{LLM} + (supplementary\ details\ from\ R_{KG})$$

The LLM can adjust its response by giving priority to high-confidence information from R_{LLM} while augmenting it with specific data points or clarifications from R_{KG}.

For example, if the user query is "What are the security controls in a public cloud?", R_{LLM} might provide a overview, while R_{KG} can include specific references to NIST guidelines and publications, such as NIST SP 800-146, detailing security recommendations for public cloud deployment models.

The end-to-end architecture and process has depicted in the below system architecture (refer Fig-4) to represent how all three sub systems interact and compliment with each other.

Fig-4: System architecture



V. Experimentation and Results

In this study, we conducted an empirical evaluation of our proposed framework using the publicly available NIST Cloud Computing Standards Roadmap PDF document [19].

The document was transformed into a graph-based representation, with nodes and relationships structured according to ontological mapping principles. This approach allowed for a more nuanced analysis of the document's inherent structure and content relationships.

We applied the PageRank algorithm to the resulting graph to identify nodes of high relevance and importance within the document's structure. The PageRank scores provided a quantitative measure of each node's relative significance, enabling us to prioritize the most influential components of the documentation. Subsequently, we extracted the relationships associated with these high-ranking nodes to create a specialized dataset. Refer Figure 5

Id	Node name	PageRank
120	Nist	3.59
399	Open Cloud standards incubator	1.82
838	Cloud management working group	1.71
291	Standard gaps	1.64

Figure-5: Top page ranked nodes from knowledge graph.

The specialized dataset generated form high ranked nodes is then utilized for fine-tuning a large language model, specifically the GPT-2 architecture. The fine-tuning process was designed to adapt the model to the specific domain and structure of the NIST documentation.

The performance of the fine-tuned GPT-2 model was evaluated using a series of text generation prompts for highly relevant content. There are 3 types of tests conducted with different percentage for highly relevant content to prove out the improvement in accuracy while also saving cost by reducing the number of tokens for highly relevant and sought content. Our results indicate a remarkable improvement in the model's accuracy and relevance when generating responses related to the NIST documentation content. The fine-tuned model demonstrated an enhanced ability to capture and reproduce the nuances and technical specifics of the source material.

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

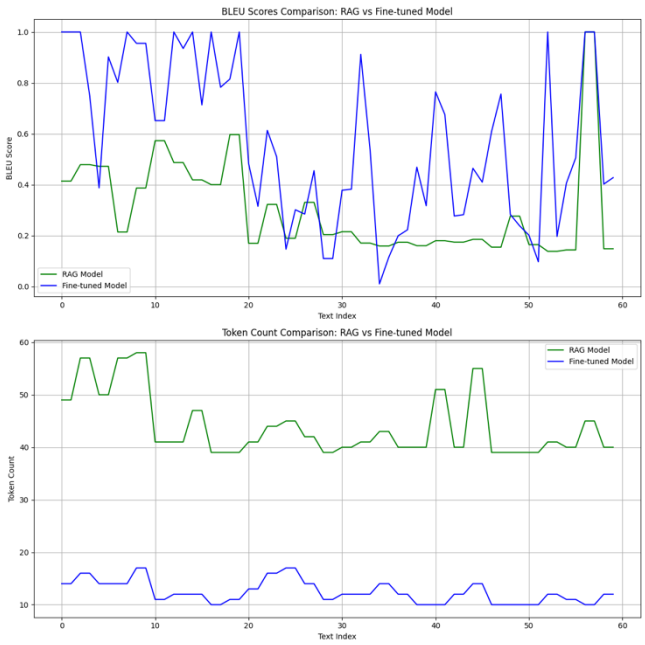


Figure-6: Graph plot of BLEU score and token count for fine-tuned Vs general model for 50% highly relevant content.

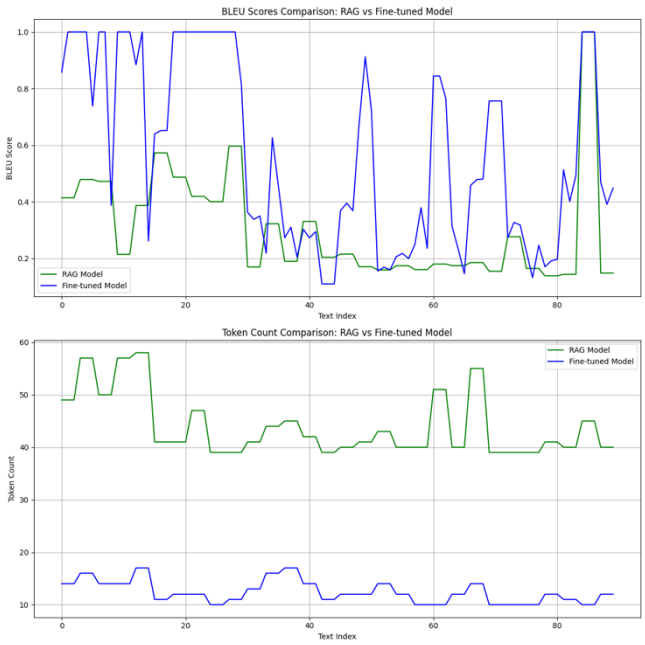


Figure-7: Graph plot of BLEU score and token count for fine-tuned Vs general model for 66% highly relevant content.

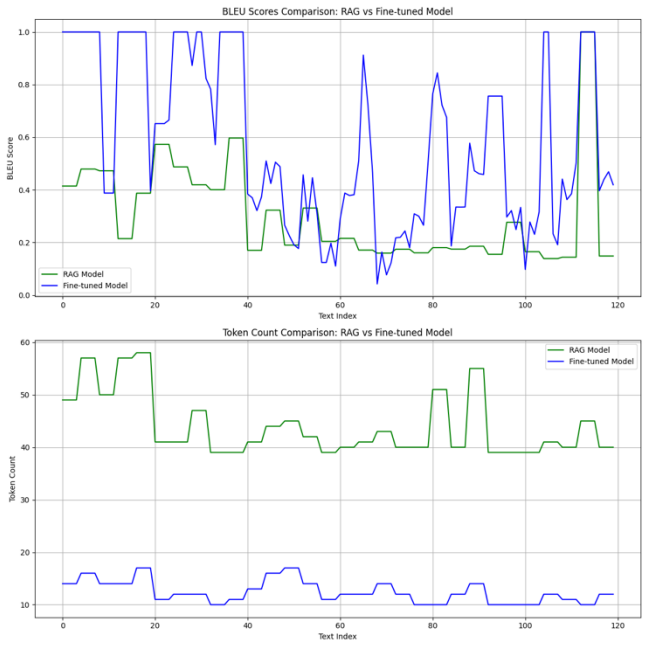


Figure-8: Graph plot of BLEU score and token count for fine-tuned Vs general model for 75% highly relevant content.

These findings in figures 6,7 and 8 suggest that our graph-based approach to document analysis, combined with targeted fine-tuning of large language models, can measurably enhance the model's performance in domain-specific tasks by 90% from BLEU score average of 0.30 to 0.57 while also reducing token count by 71.8% from average token count of 44 to 12.4 to save cost, compared to KB based RAG approach. The improved accuracy in text generation indicates a successful transfer of knowledge from the graph structure to the language model, potentially offering new avenues for automated document understanding and information retrieval in technical domains.

In summary, the proposed framework offers several significant advantages for domain-specific LLM applications:

A. Reduced Redundancy:

By fine-tuning the LLM with high ranked nodes the framework eliminates the need to repeatedly retrieve the same frequently accessed data from the knowledge graph and send to LLM for common queries used to retrieve frequently accesses nodes. This reduction in redundant operations improves overall system efficiency and reduces input token size to LLM, thereby reducing cost.

B. Incremental Knowledge Enhancement:

The automated fine-tuning process continuously updates the LLM with relevant domain knowledge, resulting in progressively more accurate and reliable model responses over time.

C. Simplified Knowledge Graph:

As highly ranked node information is incorporated into the LLM through fine-tuning, the corresponding nodes are removed from the knowledge graph. This simplification of

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

the graph structure can lead to improved retrieval speed and cost reduction in graph database.

D. Adaptive Performance:

The framework's ability to identify and incorporate highly sought information allows the LLM to adapt its performance to the evolving needs and interests within the specific domain.

E. Comprehensive Coverage:

By combining a fine-tuned LLM with a knowledge graph, the system can effectively handle both common and rare queries, ensuring comprehensive domain coverage.

To model the relationship between advantages of continuous training, accuracy improvement, and token size reduction in retrieval augmented generation using knowledge graphs, the following algebraic formulas can be used.

These formulas capture the logarithmic increase in accuracy as more high-ranking nodes are trained, the exponential decay in token size reduction, and the relationship between token size reduction and accuracy improvement. The coefficients such as (k_1, k_2, k_3) would need to be determined empirically based on the specific implementation and dataset.

1) Accuracy improvement:

$$A = A_0 + K_1.\log(1 + N)$$

Where:

A = Current accuracy
A_0 = Initial accuracy
k_1 = Accuracy improvement coefficient
N = Number of highest-ranked nodes trained

2) Token size reduction:

$$T = T_0 * (1 - k_2 * (1 - e^{-rN}))$$

Where:

T = Current token size
T_0 = Initial token size
k_2 = Maximum proportional reduction in token size
r = Rate of token size reduction
N = Number of highest-ranked nodes trained

3) Relationship between accuracy and token size:

$$A = A_0 + K_3 * \frac{(T_0 - T)}{T_0}$$

Where:

A = Current accuracy
A_0 = Initial accuracy
k_3 = Coefficient relating token reduction to accuracy improvement
T_0 = Initial token size
T = Current token size

VI. Conclusion and Future Work

This paper has presented a novel framework for automated fine-tuning of domain-specific Large Language Models with highly sought information on a graph-based Retrieval-Augmented Generation. By integrating Knowledge Graph Generation, an Automated Incremental Fine-tuning System, and a Domain-based Information Retrieval mechanism, the framework offers a robust solution for enhancing LLM performance in specialized domains in an incremental manner based on highly relevant information.

The proposed approach addresses key challenges in domain-specific AI applications, such as maintaining up-to-date knowledge, efficient information retrieval, and adaptive learning. By automating the fine-tuning process based on highly ranked, relevant and sought information and simplifying the knowledge graph structure, the framework promises improved efficiency, accuracy, and adaptability in domain-specific LLM applications.

Future work could focus on evaluating the framework's performance across various domains, optimizing the fine-tuning frequency, and exploring methods to handle conflicting or updating information in the knowledge graph. Additionally, investigating the scalability of this approach for very large domains or multi-domain applications could provide valuable insights for broader implementation.

References

[1] Ji, S., Pan, S., Cambria, E., Marttinen, P., Yu, P.S.: A survey on knowledge graphs: representation, acquisition, and applications. IEEE Trans. Neural Netw. Learn. Syst. 1–21 (2021). https://doi.org/10.1109/TNNLS.2021.3070843

[2] Vamsi Krishna Kommineni, Birgitta König-Ries, Sheeba Samuel: From human experts to machines: An LLM supported approach to ontology and knowledge graph construction arXiv:2407.06564v1 (2024)

[3] Diego Sanmartin: KG-RAG: Bridging the Gap Between Knowledge and Creativity arXiv:2405.12035 (2024)

[4] Verma, S., Bhatia, R., Harit, S. et al. Scholarly knowledge graphs through structuring scholarly communication: a review. Complex Intell. Syst. 9, 1059–1095 (2023). https://doi.org/10.1007/s40747-022-00806-6

[5] TylerProcko: Graph Retrieval-Augmented Generation for Large Language Models: A Survey (2024) https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4895062

[6] Rossanez, A., dos Reis, J.C., Torres, R.d.S. et al. KGen: a knowledge graph generator from biomedical scientific literature. BMC Med Inform Decis Mak 20 (Suppl 4), 314 (2020). https://doi.org/10.1186/s12911-020-01341-5

[7] Miao Fan, Qiang Zhou, Emily Chang, and Thomas Fang Zheng. 2014. Transition-based Knowledge Graph Embedding with Relational Mapping Properties. In Proceedings of the 28th Pacific Asia Conference on Language, Information and Computing, pages 328–337 https://aclanthology.org/Y14-1039

[8] Neelakantan, A., Chang, M.: Inferring missing entity type instances for knowledge base completion: new dataset and methods. In: NAACL, pp.515–525 (2015). https://doi.org/10.3115/v1/n15-1054

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.

[9] Zhao, Y., Zhang, A., Xie, R., Liu, K., Wang, X.: Connecting embeddings for knowledge graph entity typing. In: ACL, pp. 6419–6428 (2020). https://doi.org/10.18653/v1/2020.acl-main.572

[10] Yang, B., Yih, W., He, X., Gao, J., & Deng, L. (2014). Embedding Entities and Relations for Learning and Inference in Knowledge Bases. CoRR, abs/1412.6575.

[11] Changsung Moon, Paul Jones, and Nagiza F. Samatova. 2017. Learning Entity Type Embeddings for Knowledge Graph Completion. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM '17). Association for Computing Machinery, New York, NY, USA, 2215–2218. https://doi.org/10.1145/3132847.3133095

[12] Zhou D, Zhou B, Zheng Z, Soylu A, Savkovic O, Kostylev E and Kharlamov E. ScheRe (2022): Schema Reshaping for Enhancing Knowledge Graph Construction. Proceedings of the 31st ACM International Conference on Information & Knowledge Management. (5074-5078). https://doi.org/10.1145/3511808.3557214

[13] Chen, S., Wang, J., Jiang, F., & Lin, C.-Y. (2020). Improving Entity Linking by Modeling Latent Entity Type Information. Proceedings of the AAAI Conference on Artificial Intelligence, 34(05), 7529-7537. https://doi.org/10.1609/aaai.v34i05.6251

[14] Larry Page and Sergey Brin , The Anatomy of a Large-Scale Hypertextual Web Search Engine http://infolab.stanford.edu/~backrub/google.html

[15] Bundit Manaskasemsak and Arnon Rungsawang (2005): An Efficient Partition-Based Parallel PageRank Algorithm http://delab.csd.auth.gr/~dimitris/courses/ir_spring06/page_rank_computing/01531136.pdf

[16] DAVID F. GLEICH (2014): Page Rank Beyond the Web https://arxiv.org/pdf/1407.5107.pdf

[17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401

[18] Paulheim, H., Bizer, C. (2013). Type Inference on Noisy RDF Data. In:, et al. The Semantic Web – ISWC 2013. ISWC 2013. Lecture Notes in Computer Science, vol 8218. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-41335-3_32

[19] NIST Cloud Computing Standards Roadmap. http://dx.doi.org/10.6028/NIST.SP.500-291r2

[20] Qi Zhu, Hao Wei, Bunyamin Sisman, Da Zheng, Christos Faloutsos, Xin Luna Dong, and Jiawei Han. 2020. Collective Multi-type Entity Alignment Between Knowledge Graphs. In Proceedings of The Web Conference 2020 (WWW '20). Association for Computing Machinery, New York, NY.

Authorized licensed use limited to: VTU Consortium. Downloaded on March 04,2025 at 04:25:35 UTC from IEEE Xplore. Restrictions apply.